Imports and list files

```
# Cell 1 — Imports and show current files
import os
import pandas as pd
from pathlib import Path

print("Working directory:", os.getcwd())
print("\nFiles in the current working directory (first 50):")
for i, f in enumerate(sorted(os.listdir(".."))[:50], 1):
    print(f"{i:2d}. {f}")
```

```
Working directory: /content

Files in the current working directory (first 50):
 1. .config
 2. Fake.csv
 3. True.csv
 4. sample_data
```

Check for Fake.csv and True.csv

```
# Cell 2 — Check for dataset files
fake_path = Path("Fake.csv")
true_path = Path("True.csv")

print("Checking for dataset files...")
print("Fake.csv exists:", fake_path.exists())
print("True.csv exists:", true_path.exists())

if not fake_path.exists():
    print("\nIf Fake.csv is missing: Upload it (from Kaggle) using the Colab file browser (left panel).")
if not true_path.exists():
    print("\nIf True.csv is missing: Upload it (the True.csv file from Kaggle) or rename your real-news CSV to True.csv.")
```

```
Checking for dataset files...
Fake.csv exists: True
True.csv exists: True
```

Robust CSV loader (detects best title/text columns)

```
# Cell 3 — Robust loader function (detects likely title/text columns)
def load_csv_best(filepath, cols_prefer=("title","text","headline","content")):
    df = pd.read_csv(filepath, low_memory=False)
    cols = [c.lower() for c in df.columns]
    title_col = None
    text_col = None
    for cand in cols_prefer:
        if cand in cols:
            title_col = df.columns[cols.index(cand)]
            break
    for cand in ("text","content","article","body"):
        if cand in cols:
            text_col = df.columns[cols.index(cand)]
            break
    if title_col is None:
        title_col = df.columns[0]
    if text_col is None:
        text_col = df.columns[1] if len(df.columns) > 1 else df.columns[0]
    return df, title_col, text_col

print("Loader ready. Use load_csv_best('Fake.csv') or load_csv_best('True.csv')")
```

```
Loader ready. Use load_csv_best('Fake.csv') or load_csv_best('True.csv')
```

Load and preview Fake.csv and True.csv

```
# Cell 4 — Load the files if they exist and show a small preview
loaded = {}
```

```
if fake_path.exists():
    df_fake, fake_title_col, fake_text_col = load_csv_best(fake_path)
    print("\nLoaded Fake.csv with columns:", list(df_fake.columns)[:10])
    print("Chosen title column:", fake_title_col, "| text column:", fake_text_col)
    display(df_fake[[fake_title_col, fake_text_col]].head(3))
    loaded['fake'] = (df_fake, fake_title_col, fake_text_col)
else:
    print("\nFake.csv not found. Upload Fake.csv and re-run this cell.")

if true_path.exists():
    df_true, true_title_col, true_text_col = load_csv_best(true_path)
    print("\nLoaded True.csv with columns:", list(df_true.columns)[:10])
    print("Chosen title column:", true_title_col, "| text column:", true_text_col)
    display(df_true[[true_title_col, true_text_col]].head(3))
    loaded['true'] = (df_true, true_title_col, true_text_col)
else:
    print("\nTrue.csv not found. If you have a real-news file, upload it and re-run this cell.")
```

```
Loaded Fake.csv with columns: ['title', 'text', 'subject', 'date']
Chosen title column: title | text column: text
```

	title	text
0	Donald Trump Sends Out Embarrassing New Year'...	Donald Trump just couldn t wish all Americans ...
1	Drunk Bragging Trump Staffer Started Russian ...	House Intelligence Committee Chairman Devin Nu...
2	Sheriff David Clarke Becomes An Internet Joke...	On Friday, it was revealed that former Milwauk...

```
Loaded True.csv with columns: ['title', 'text', 'subject', 'date']
Chosen title column: title | text column: text
```

	title	text
0	As U.S. budget fight looms, Republicans flip t...	WASHINGTON (Reuters) - The head of a conservat...
1	U.S. military to accept transgender recruits o...	WASHINGTON (Reuters) - Transgender people will...
2	Senior U.S. Republican senator: 'Let Mr. Muell...	WASHINGTON (Reuters) - The special counsel inv...

Standardise columns, label, combine, shuffle and save

```
# Cell 5 — Prepare consistent columns and combine
if 'fake' in loaded:
    df_fake, ftc, ftxt = loaded['fake']
    df_fake_pre = pd.DataFrame({
        'title': df_fake[ftc].astype(str),
        'text': df_fake[ftxt].astype(str),
        'label': 0   # 0 = Fake
    })
    print("Prepared fake dataset rows:", len(df_fake_pre))
else:
    df_fake_pre = None

if 'true' in loaded:
    df_true, ttc, ttxt = loaded['true']
    df_true_pre = pd.DataFrame({
        'title': df_true[ttc].astype(str),
        'text': df_true[ttxt].astype(str),
        'label': 1   # 1 = Real
    })
    print("Prepared true dataset rows:", len(df_true_pre))
else:
    df_true_pre = None

if df_fake_pre is not None and df_true_pre is not None:
    combined = pd.concat([df_fake_pre, df_true_pre], axis=0).sample(frac=1, random_state=42).reset_index(drop=True)
    print("Combined dataset shape:", combined.shape)
    display(combined.head())
    combined.to_csv("combined_news.csv", index=False)
    print("Saved combined dataset to combined_news.csv")
elif df_fake_pre is not None:
    df_fake_pre.to_csv("fake_prepared.csv", index=False)
    print("Saved prepared fake-only file to fake_prepared.csv. Add True.csv later and re-run to combine.")
else:
    print("No data prepared. Please upload Fake.csv and True.csv and re-run the cells.")
```

```
Prepared fake dataset rows: 23481
Prepared true dataset rows: 21417
Combined dataset shape: (44898, 3)
```

	title	text	label
0	Ben Stein Calls Out 9th Circuit Court: Committ...	21st Century Wire says Ben Stein, reputable pr...	0
1	Trump drops Steve Bannon from National Securit...	WASHINGTON (Reuters) - U.S. President Donald T...	1
2	Puerto Rico expects U.S. to lift Jones Act shi...	(Reuters) - Puerto Rico Governor Ricardo Rosse...	1
3	OOPS: Trump Just Accidentally Confirmed He Le...	On Monday, Donald Trump once again embarrassed...	0
4	Donald Trump heads for Scotland to reopen a go...	GLASGOW, Scotland (Reuters) - Most U.S. presid...	1

```
Saved combined dataset to combined_news.csv
```

Quick checks: class balance and basic stats

```
# Cell 6 — Quick dataset checks (run after you have combined_news.csv)
import pandas as pd
if Path("combined_news.csv").exists():
    df = pd.read_csv("combined_news.csv")
    print("Combined shape:", df.shape)
    print("\nClass distribution:\n", df['label'].value_counts(normalize=False))
    print("\nExample titles:")
    display(df['title'].head(5))
else:
    print("combined_news.csv not found. If you saved fake_prepared.csv only, upload True.csv and re-run the combine step.")
```

```
Combined shape: (44898, 3)

Class distribution:
 label
0    23481
1    21417
Name: count, dtype: int64

Example titles:
```

	title
0	Ben Stein Calls Out 9th Circuit Court: Committ...
1	Trump drops Steve Bannon from National Securit...
2	Puerto Rico expects U.S. to lift Jones Act shi...
3	OOPS: Trump Just Accidentally Confirmed He Le...
4	Donald Trump heads for Scotland to reopen a go...

dtype: object

Text Preprocessing + TF-IDF Vectorisation

Import libraries and load dataset

```
# Cell 1 — Import libraries and load the combined dataset
import pandas as pd
import numpy as np
import re
import nltk
nltk.download('stopwords')
from nltk.corpus import stopwords

# Load the dataset (the file created earlier)
df = pd.read_csv("combined_news.csv")
print("Dataset loaded successfully!")
print("Shape:", df.shape)
df.head()
```

```
[nltk_data] Downloading package stopwords to /root/nltk_data...
[nltk_data]   Unzipping corpora/stopwords.zip.
Dataset loaded successfully!
Shape: (44898, 3)
```

	title	text	label
0	Ben Stein Calls Out 9th Circuit Court: Committ...	21st Century Wire says Ben Stein, reputable pr...	0
1	Trump drops Steve Bannon from National Securit...	WASHINGTON (Reuters) - U.S. President Donald T...	1
2	Puerto Rico expects U.S. to lift Jones Act shi...	(Reuters) - Puerto Rico Governor Ricardo Rosse...	1
3	OOPS: Trump Just Accidentally Confirmed He Le...	On Monday, Donald Trump once again embarrassed...	0
4	Donald Trump heads for Scotland to reopen a go...	GLASGOW, Scotland (Reuters) - Most U.S. presid...	1

Next steps: Generate code with df New interactive sheet

Check for missing values and basic info

```
# Cell 2 — Quick data quality check
print("Missing values before cleaning:")
print(df.isnull().sum())

# Drop rows with missing text or title
df = df.dropna(subset=['text', 'title']).reset_index(drop=True)

print("\nAfter removing missing values:")
print(df.isnull().sum())

print("\nClass balance:")
print(df['label'].value_counts())
```

```
Missing values before cleaning:
title    0
text     0
label    0
dtype: int64

After removing missing values:
title    0
text     0
label    0
dtype: int64

Class balance:
label
0    23481
1    21417
Name: count, dtype: int64
```

Clean and tokenise the text

```
# Cell 3 — Define text-cleaning function
from string import punctuation

stop_words = set(stopwords.words('english'))

def clean_text(text):
    text = str(text).lower()                        # Lowercase
    text = re.sub(r'http\S+|www.\S+', '', text)      # Remove links
    text = re.sub(r'[^a-z\s]', '', text)             # Keep only letters
    text = re.sub(r'\s+', ' ', text).strip()         # Remove extra spaces
    words = [w for w in text.split() if w not in stop_words]
    return " ".join(words)

# Apply to the text column
df['clean_text'] = df['text'].apply(clean_text)

print("Cleaning complete! Example:")
print(df['clean_text'].head(3))
```

```
Cleaning complete! Example:
0    st century wire says ben stein reputable profe...
1    washington reuters us president donald trump r...
```

```
2    reuters puerto rico governor ricardo rossello ...
Name: clean_text, dtype: object
```

Token count and text length

```
# Cell 4 — Optional: check word counts and distribution
df['word_count'] = df['clean_text'].apply(lambda x: len(x.split()))
print(df['word_count'].describe())
```

```
count    44898.000000
mean       228.705377
std        199.255169
min          0.000000
25%        115.000000
50%        201.000000
75%        286.000000
max       4841.000000
Name: word_count, dtype: float64
```

Split into training and test sets

```
# Cell 5 — Split the data into training and testing sets
from sklearn.model_selection import train_test_split

X = df['clean_text']
y = df['label']

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42, stratify=y
)

print("Training samples:", X_train.shape[0])
print("Testing samples:", X_test.shape[0])
```

```
Training samples: 35918
Testing samples: 8980
```

TF-IDF Vectorisation

```
# Cell 6 — TF-IDF vectorisation
from sklearn.feature_extraction.text import TfidfVectorizer

tfidf_vectorizer = TfidfVectorizer(
    max_features=5000,          # limit to 5k most important words
    ngram_range=(1,2),          # unigrams and bigrams
    stop_words='english'
)

X_train_tfidf = tfidf_vectorizer.fit_transform(X_train)
X_test_tfidf = tfidf_vectorizer.transform(X_test)

print("TF-IDF transformation complete!")
print("Train matrix shape:", X_train_tfidf.shape)
print("Test matrix shape:", X_test_tfidf.shape)
```

```
TF-IDF transformation complete!
Train matrix shape: (35918, 5000)
Test matrix shape: (8980, 5000)
```

Save vectoriser for later use

```
# Cell 7 — Save the TF-IDF vectoriser for reuse in the real-time model
import joblib
joblib.dump(tfidf_vectorizer, "tfidf_vectorizer.pkl")
print("Vectoriser saved as tfidf_vectorizer.pkl")
```

```
Vectoriser saved as tfidf_vectorizer.pkl
```

Import libraries and load TF-IDF data

```
# Cell 1 — Import libraries and reload TF-IDF data (if needed)
import joblib
import numpy as np
from sklearn.metrics import accuracy_score, classification_report, confusion_matrix

# Load previously saved TF-IDF vectorizer (if needed later)
vectorizer = joblib.load("tfidf_vectorizer.pkl")

print("Libraries loaded and vectorizer ready.")
```

```
Libraries loaded and vectorizer ready.
```

Train Logistic Regression

```
# Cell 2 — Train Logistic Regression (main model)
from sklearn.linear_model import LogisticRegression

lr_model = LogisticRegression(max_iter=1000)
lr_model.fit(X_train_tfidf, y_train)

y_pred_lr = lr_model.predict(X_test_tfidf)

print("Logistic Regression Results:")
print("Accuracy:", accuracy_score(y_test, y_pred_lr))
print("\nClassification Report:\n", classification_report(y_test, y_pred_lr))
```

```
Logistic Regression Results:
Accuracy: 0.9898663697104677

Classification Report:
               precision    recall  f1-score   support

           0       0.99      0.99      0.99      4696
           1       0.99      0.99      0.99      4284

    accuracy                           0.99      8980
   macro avg       0.99      0.99      0.99      8980
weighted avg       0.99      0.99      0.99      8980
```

Train Naïve Bayes

```
# Cell 3 — Train Multinomial Naive Bayes (lightweight comparison)
from sklearn.naive_bayes import MultinomialNB

nb_model = MultinomialNB()
nb_model.fit(X_train_tfidf, y_train)

y_pred_nb = nb_model.predict(X_test_tfidf)

print("Naive Bayes Results:")
print("Accuracy:", accuracy_score(y_test, y_pred_nb))
print("\nClassification Report:\n", classification_report(y_test, y_pred_nb))
```

```
Naive Bayes Results:
Accuracy: 0.9496659242761692

Classification Report:
               precision    recall  f1-score   support

           0       0.95      0.95      0.95      4696
           1       0.94      0.95      0.95      4284

    accuracy                           0.95      8980
   macro avg       0.95      0.95      0.95      8980
weighted avg       0.95      0.95      0.95      8980
```

Train Linear SVM

```
# Cell 4 — Train Linear SVM (Support Vector Machine)
from sklearn.svm import LinearSVC

svm_model = LinearSVC()
```

```
svm_model.fit(X_train_tfidf, y_train)

y_pred_svm = svm_model.predict(X_test_tfidf)

print("SVM Results:")
print("Accuracy:", accuracy_score(y_test, y_pred_svm))
print("\nClassification Report:\n", classification_report(y_test, y_pred_svm))
```

```
SVM Results:
Accuracy: 0.9962138084632517

Classification Report:
               precision    recall  f1-score   support

           0       1.00      1.00      1.00      4696
           1       0.99      1.00      1.00      4284

    accuracy                           1.00      8980
   macro avg       1.00      1.00      1.00      8980
weighted avg       1.00      1.00      1.00      8980
```

Compare all three models

```
# Cell 5 — Compare model performance
acc_lr = accuracy_score(y_test, y_pred_lr)
acc_nb = accuracy_score(y_test, y_pred_nb)
acc_svm = accuracy_score(y_test, y_pred_svm)

print("\nModel Comparison:")
print(f"Logistic Regression: {acc_lr:.4f}")
print(f"Naive Bayes:         {acc_nb:.4f}")
print(f"Linear SVM:          {acc_svm:.4f}")

best_model_name = max(
    {"Logistic Regression": acc_lr, "Naive Bayes": acc_nb, "Linear SVM": acc_svm},
    key=lambda k: {"Logistic Regression": acc_lr, "Naive Bayes": acc_nb, "Linear SVM": acc_svm}[k]
)
print("\nBest Model Based on Accuracy:", best_model_name)
```

```
Model Comparison:
Logistic Regression: 0.9899
Naive Bayes:         0.9497
Linear SVM:          0.9962

Best Model Based on Accuracy: Linear SVM
```

Save the best model

```
# Cell 6 — Save the best performing model for later (to connect with NewsAPI)
if best_model_name == "Logistic Regression":
    best_model = lr_model
elif best_model_name == "Naive Bayes":
    best_model = nb_model
else:
    best_model = svm_model

joblib.dump(best_model, "fake_news_model.pkl")
print(f"Best model ({best_model_name}) saved as fake_news_model.pkl")
```

```
Best model (Linear SVM) saved as fake_news_model.pkl
```

Test real-time prediction simulation

```
# Cell 7 — Simulate real-time prediction on random samples
import random, time

sample_articles = random.sample(list(X_test), 5)

for article in sample_articles:
    processed = vectorizer.transform([article])
    start = time.time()
    prediction = best_model.predict(processed)[0]
```

```
    end = time.time()
    label = "Fake" if prediction == 0 else "Real"
    print(f"\nNews: {article[:100]}...")
    print(f"Prediction: {label} | Time taken: {end - start:.4f} seconds")
```

```
News: group imams tried bring form shariah light texas met unlikely foe irving mayor beth van duynevan duy...
Prediction: Fake | Time taken: 0.0004 seconds

News: st century wire says defendants federal case nevada rancher cliven bundy sons group supporters key w...
Prediction: Fake | Time taken: 0.0008 seconds

News: washington reuters us attorney general jeff sessions tuesday released memo restricting justice depar...
Prediction: Real | Time taken: 0.0003 seconds

News: love know thoughts think sort unreal company forced via social media drop name change refugee dress ...
Prediction: Fake | Time taken: 0.0003 seconds

News: washington reuters white house plans ask us congress third round disaster aid midnovember costs cont...
Prediction: Real | Time taken: 0.0003 seconds
```

Connect to NewsAPI for live data

```
# Cell 8 — Real-time live news detection via NewsAPI
import requests

api_key = "e0e70dade33d46599e66beafd6658eed"  # replace with your key from https://newsapi.org
url = f"https://newsapi.org/v2/top-headlines?language=en&pageSize=10&apiKey={api_key}"

data = requests.get(url).json()

print("Fetching live headlines...\n")

for article in data.get("articles", []):
    title = article.get("title", "")
    if title:
        processed = vectorizer.transform([title])
        start = time.time()
        prediction = best_model.predict(processed)[0]
        end = time.time()
        label = "Fake" if prediction == 0 else "Real"
        print(f"{title}\n→ Prediction: {label} | Time: {end - start:.4f}s\n")
```

```
Fetching live headlines...

Laden Iranian ships depart Chinese port tied to key military chemicals - The Washington Post
→ Prediction: Fake | Time: 0.0002s

Kentucky Basketball loses to Florida on Senior Day: 3 things to know and postgame boos - A Sea Of Blue
→ Prediction: Fake | Time: 0.0002s

Trump says Iran at fault for strike on girls school - Politico
→ Prediction: Fake | Time: 0.0002s

Times the U.S. has installed a foreign leader, as Trump zeroes in on Iran - Axios
→ Prediction: Fake | Time: 0.0002s

Dan Hurley Loses His Mind, Gets Ejected With One Second Left Against Marquette - OutKick
→ Prediction: Fake | Time: 0.0002s

Shedding light on how Epstein used visits to Interlochen to target girls - NPR
→ Prediction: Fake | Time: 0.0002s

Live results and analysis for UFC 326: Holloway vs. Oliveira 2 - ESPN
→ Prediction: Fake | Time: 0.0002s

Health officials confirm measles case in visitor to Hawaii - Honolulu Star-Advertiser
→ Prediction: Fake | Time: 0.0003s

Rapper-politician Balendra Shah on course to be Nepal's next prime minister - BBC
→ Prediction: Fake | Time: 0.0002s

Enormous 3D map of the universe shows brilliant 'sea of light' near the cosmic dawn - Live Science
→ Prediction: Fake | Time: 0.0002s
```

Install and Import Libraries for Deep Learning

```
# Cell 1 — Install and import deep learning libraries
!pip install tensorflow

import tensorflow as tf
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Dense, Embedding, Conv1D, GlobalMaxPooling1D, Dropout
from tensorflow.keras.preprocessing.text import Tokenizer
from tensorflow.keras.preprocessing.sequence import pad_sequences
from sklearn.model_selection import train_test_split
from sklearn.metrics import classification_report, accuracy_score
```

```
Requirement already satisfied: tensorflow in /usr/local/lib/python3.12/dist-packages (2.19.0)
Requirement already satisfied: absl-py>=1.0.0 in /usr/local/lib/python3.12/dist-packages (from tensorflow) (1.4.0)
Requirement already satisfied: astunparse>=1.6.0 in /usr/local/lib/python3.12/dist-packages (from tensorflow) (1.6.3)
Requirement already satisfied: flatbuffers>=24.3.25 in /usr/local/lib/python3.12/dist-packages (from tensorflow) (25.12.19)
Requirement already satisfied: gast!=0.5.0,!=0.5.1,!=0.5.2,>=0.2.1 in /usr/local/lib/python3.12/dist-packages (from tensorflo
Requirement already satisfied: google-pasta>=0.1.1 in /usr/local/lib/python3.12/dist-packages (from tensorflow) (0.2.0)
Requirement already satisfied: libclang>=13.0.0 in /usr/local/lib/python3.12/dist-packages (from tensorflow) (18.1.1)
Requirement already satisfied: opt-einsum>=2.3.2 in /usr/local/lib/python3.12/dist-packages (from tensorflow) (3.4.0)
Requirement already satisfied: packaging in /usr/local/lib/python3.12/dist-packages (from tensorflow) (26.0)
Requirement already satisfied: protobuf!=4.21.0,!=4.21.1,!=4.21.2,!=4.21.3,!=4.21.4,!=4.21.5,<6.0.0dev,>=3.20.3 in /usr/local
Requirement already satisfied: requests<3,>=2.21.0 in /usr/local/lib/python3.12/dist-packages (from tensorflow) (2.32.4)
Requirement already satisfied: setuptools in /usr/local/lib/python3.12/dist-packages (from tensorflow) (75.2.0)
Requirement already satisfied: six>=1.12.0 in /usr/local/lib/python3.12/dist-packages (from tensorflow) (1.17.0)
Requirement already satisfied: termcolor>=1.1.0 in /usr/local/lib/python3.12/dist-packages (from tensorflow) (3.3.0)
Requirement already satisfied: typing-extensions>=3.6.6 in /usr/local/lib/python3.12/dist-packages (from tensorflow) (4.15.0)
Requirement already satisfied: wrapt>=1.11.0 in /usr/local/lib/python3.12/dist-packages (from tensorflow) (2.1.1)
Requirement already satisfied: grpcio<2.0,>=1.24.3 in /usr/local/lib/python3.12/dist-packages (from tensorflow) (1.78.0)
Requirement already satisfied: tensorboard~=2.19.0 in /usr/local/lib/python3.12/dist-packages (from tensorflow) (2.19.0)
Requirement already satisfied: keras>=3.5.0 in /usr/local/lib/python3.12/dist-packages (from tensorflow) (3.10.0)
Requirement already satisfied: numpy<2.2.0,>=1.26.0 in /usr/local/lib/python3.12/dist-packages (from tensorflow) (2.0.2)
Requirement already satisfied: h5py>=3.11.0 in /usr/local/lib/python3.12/dist-packages (from tensorflow) (3.15.1)
Requirement already satisfied: ml-dtypes<1.0.0,>=0.5.1 in /usr/local/lib/python3.12/dist-packages (from tensorflow) (0.5.4)
Requirement already satisfied: wheel<1.0,>=0.23.0 in /usr/local/lib/python3.12/dist-packages (from astunparse>=1.6.0->tensorf
Requirement already satisfied: rich in /usr/local/lib/python3.12/dist-packages (from keras>=3.5.0->tensorflow) (13.9.4)
Requirement already satisfied: namex in /usr/local/lib/python3.12/dist-packages (from keras>=3.5.0->tensorflow) (0.1.0)
Requirement already satisfied: optree in /usr/local/lib/python3.12/dist-packages (from keras>=3.5.0->tensorflow) (0.19.0)
Requirement already satisfied: charset_normalizer<4,>=2 in /usr/local/lib/python3.12/dist-packages (from requests<3,>=2.21.0-
Requirement already satisfied: idna<4,>=2.5 in /usr/local/lib/python3.12/dist-packages (from requests<3,>=2.21.0->tensorflow)
Requirement already satisfied: urllib3<3,>=1.21.1 in /usr/local/lib/python3.12/dist-packages (from requests<3,>=2.21.0->tenso
Requirement already satisfied: certifi>=2017.4.17 in /usr/local/lib/python3.12/dist-packages (from requests<3,>=2.21.0->tenso
Requirement already satisfied: markdown>=2.6.8 in /usr/local/lib/python3.12/dist-packages (from tensorboard~=2.19.0->tensorfl
Requirement already satisfied: tensorboard-data-server<0.8.0,>=0.7.0 in /usr/local/lib/python3.12/dist-packages (from tensorb
Requirement already satisfied: werkzeug>=1.0.1 in /usr/local/lib/python3.12/dist-packages (from tensorboard~=2.19.0->tensorfl
Requirement already satisfied: markupsafe>=2.1.1 in /usr/local/lib/python3.12/dist-packages (from werkzeug>=1.0.1->tensorboar
Requirement already satisfied: markdown-it-py>=2.2.0 in /usr/local/lib/python3.12/dist-packages (from rich->keras>=3.5.0->ten
Requirement already satisfied: pygments<3.0.0,>=2.13.0 in /usr/local/lib/python3.12/dist-packages (from rich->keras>=3.5.0->t
Requirement already satisfied: mdurl~=0.1 in /usr/local/lib/python3.12/dist-packages (from markdown-it-py>=2.2.0->rich->keras
```

Prepare Text for Deep Learning

```
# Cell 2 — Tokenization

max_words = 20000
max_length = 200

tokenizer = Tokenizer(num_words=max_words)
tokenizer.fit_on_texts(df['text'])

sequences = tokenizer.texts_to_sequences(df['text'])

X = pad_sequences(sequences, maxlen=max_length)

y = df['label']
```

Train Test Split

```
# Cell 3 — Split dataset

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)

print("Training size:", len(X_train))
print("Testing size:", len(X_test))
```

```
Training size: 35918
Testing size: 8980
```

Build CNN Model

```
# CNN Model (fixed version)

model = Sequential()

model.add(Embedding(input_dim=max_words,
                    output_dim=128,
                    input_length=max_length))

model.add(Conv1D(filters=128, kernel_size=5, activation='relu'))

model.add(GlobalMaxPooling1D())

model.add(Dense(64, activation='relu'))

model.add(Dropout(0.5))

model.add(Dense(1, activation='sigmoid'))

model.compile(
    loss='binary_crossentropy',
    optimizer='adam',
    metrics=['accuracy']
)

model.build(input_shape=(None, max_length))

model.summary()
```

Model: "sequential_1"

Layer (type)	Output Shape	Param #
embedding_1 (Embedding)	(None, 200, 128)	2,560,000
conv1d_1 (Conv1D)	(None, 196, 128)	82,048
global_max_pooling1d_1 (GlobalMaxPooling1D)	(None, 128)	0
dense_2 (Dense)	(None, 64)	8,256
dropout_1 (Dropout)	(None, 64)	0
dense_3 (Dense)	(None, 1)	65

Total params: 2,650,369 (10.11 MB)
Trainable params: 2,650,369 (10.11 MB)

Train CNN Model

```
# Cell 5 — Train model

history = model.fit(
    X_train,
    y_train,
    epochs=5,
    batch_size=128,
    validation_split=0.1
)
```

```
Epoch 1/5
253/253 ━━━━━━━━━━━━━━━━━━━━ 14s 27ms/step - accuracy: 0.8481 - loss: 0.3200 - val_accuracy: 0.9897 - val_loss: 0.0287
Epoch 2/5
253/253 ━━━━━━━━━━━━━━━━━━━━ 2s 7ms/step - accuracy: 0.9954 - loss: 0.0157 - val_accuracy: 0.9922 - val_loss: 0.0222
Epoch 3/5
253/253 ━━━━━━━━━━━━━━━━━━━━ 2s 7ms/step - accuracy: 0.9995 - loss: 0.0034 - val_accuracy: 0.9947 - val_loss: 0.0151
Epoch 4/5
253/253 ━━━━━━━━━━━━━━━━━━━━ 2s 7ms/step - accuracy: 1.0000 - loss: 8.6242e-04 - val_accuracy: 0.9953 - val_loss: 0.0152
Epoch 5/5
253/253 ━━━━━━━━━━━━━━━━━━━━ 2s 7ms/step - accuracy: 1.0000 - loss: 4.9775e-04 - val_accuracy: 0.9950 - val_loss: 0.0176
```

Evaluate Model

```
# Cell 6 — Evaluate

y_pred = (model.predict(X_test) > 0.5).astype("int32")

print("Accuracy:", accuracy_score(y_test, y_pred))

print(classification_report(y_test, y_pred))
```

```
281/281 ━━━━━━━━━━━━━━━━━━━━ 2s 5ms/step
Accuracy: 0.9944320712694877
              precision    recall  f1-score   support

           0       0.99      0.99      0.99      4710
           1       0.99      0.99      0.99      4270

    accuracy                           0.99      8980
   macro avg       0.99      0.99      0.99      8980
weighted avg       0.99      0.99      0.99      8980
```

Prepare Function for Real-Time Prediction

```
# Cell 7 — Prediction function

def predict_news(text):

    seq = tokenizer.texts_to_sequences([text])
    padded = pad_sequences(seq, maxlen=max_length)

    pred = model.predict(padded)[0][0]

    if pred > 0.5:
        return "Real"
    else:
        return "Fake"
```

Real-Time News Detection (NewsAPI)

```
# Cell 8 — Real-time detection

import requests
import time

api_key = "e0e70dade33d46599e66beafd6658eed"

url = f"https://newsapi.org/v2/top-headlines?language=en&pageSize=10&apiKey={api_key}"

data = requests.get(url).json()

print("\nLive News Detection\n")

for article in data["articles"]:

    title = article["title"]

    start = time.time()

    result = predict_news(title)

    end = time.time()
```

```
    print(title)
    print("Prediction:", result)
    print("Time:", round(end-start,4),"seconds\n")
```

```
Live News Detection

1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 308ms/step
ICE taking steps to close detention center at Fort Bliss, document shows - The Washington Post
Prediction: Fake
Time: 0.3463 seconds

1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 39ms/step
US Senate vote fails to rein in Trump war powers on Iran - BBC
Prediction: Fake
Time: 0.0746 seconds

1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 43ms/step
Judge rules companies are entitled to refunds for Trump tariffs overturned by the Supreme Court - AP News
Prediction: Fake
Time: 0.0775 seconds

1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 49ms/step
Broadcom Inc. Announces First Quarter Fiscal Year 2026 Financial Results and Quarterly Dividend - Broadcom
Prediction: Fake
Time: 0.0839 seconds

1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 37ms/step
‘SNL’ Promo: Ryan Gosling Thinks He’s Joining the Five-Timers Club, But There’s Just One Problem… - Deadline
Prediction: Fake
Time: 0.068 seconds

1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 35ms/step
Kenneth Walker III potential landing spots: Where will Super Bowl LX MVP play in 2026? - NFL.com
Prediction: Fake
Time: 0.0648 seconds

1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 43ms/step
Los Gatos 'Party Mom' found guilty after 3-month trial - SFGATE
Prediction: Fake
Time: 0.0785 seconds

1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 39ms/step
Oscar Season’s Most Burning Question: Why Not Timothée? - The Hollywood Reporter
Prediction: Fake
Time: 0.0712 seconds

1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 39ms/step
'Every day feels like a month': Iranians describe life under 'constant' US-Israeli strikes - BBC
Prediction: Fake
Time: 0.0749 seconds

1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 37ms/step
Man United's Harry Maguire gets suspended sentence over 2020 nightclub incident in Greece - AP News
Prediction: Fake
Time: 0.0706 seconds
```

DistilBert

```
# Cell 1 — Install required libraries
!pip -q install transformers datasets accelerate evaluate
```

```
━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 84.1/84.1 kB 7.4 MB/s eta 0:00:00
```

```
# Cell 2 — Imports
import os
import random
import time
import numpy as np
import pandas as pd
import torch

from sklearn.model_selection import train_test_split
from sklearn.metrics import accuracy_score, precision_recall_fscore_support, classification_report

from datasets import Dataset, DatasetDict
```

```
from transformers import (
    AutoTokenizer,
    AutoModelForSequenceClassification,
    DataCollatorWithPadding,
    TrainingArguments,
    Trainer
)

print("Torch version:", torch.__version__)
print("GPU available:", torch.cuda.is_available())
if torch.cuda.is_available():
    print("GPU name:", torch.cuda.get_device_name(0))
```

```
Torch version: 2.10.0+cu128
GPU available: True
GPU name: Tesla T4
```

set Seed

```
# Cell 3 — Reproducibility
def set_seed(seed=42):
    random.seed(seed)
    np.random.seed(seed)
    torch.manual_seed(seed)
    if torch.cuda.is_available():
        torch.cuda.manual_seed_all(seed)

set_seed(42)
print("Seed set.")
```

```
Seed set.
```

```
# Cell 4 — Load combined dataset
df = pd.read_csv("combined_news.csv")

print("Shape:", df.shape)
print(df.head())
print("\nClass balance:")
print(df["label"].value_counts())
```

```
Shape: (44898, 3)
                                               title  \
0  Ben Stein Calls Out 9th Circuit Court: Committ...
1  Trump drops Steve Bannon from National Securit...
2  Puerto Rico expects U.S. to lift Jones Act shi...
3   OOPS: Trump Just Accidentally Confirmed He Le...
4  Donald Trump heads for Scotland to reopen a go...

                                                text  label
0  21st Century Wire says Ben Stein, reputable pr...      0
1  WASHINGTON (Reuters) - U.S. President Donald T...      1
2  (Reuters) - Puerto Rico Governor Ricardo Rosse...      1
3  On Monday, Donald Trump once again embarrassed...      0
4  GLASGOW, Scotland (Reuters) - Most U.S. presid...      1

Class balance:
label
0    23481
1    21417
Name: count, dtype: int64
```

Preparing Text

```
# Cell 5 — Create text input for DistilBERT
df["title"] = df["title"].fillna("")
df["text"] = df["text"].fillna("")

# Use title + first 1000 characters of article text
df["model_text"] = (df["title"] + " " + df["text"].str[:1000]).str.strip()

# Keep only needed columns
df = df[["model_text", "label"]].copy()

print(df.head())
```

```
                                          model_text  label
0  Ben Stein Calls Out 9th Circuit Court: Committ...      0
1  Trump drops Steve Bannon from National Securit...      1
2  Puerto Rico expects U.S. to lift Jones Act shi...      1
3  OOPS: Trump Just Accidentally Confirmed He Lea...      0
4  Donald Trump heads for Scotland to reopen a go...      1
```

Train/Test split

```
# Cell 6 — Split the dataset
train_df, test_df = train_test_split(
    df,
    test_size=0.2,
    random_state=42,
    stratify=df["label"]
)

print("Train size:", train_df.shape)
print("Test size:", test_df.shape)
```

```
Train size: (35918, 2)
Test size: (8980, 2)
```

Convert to Hugging Face dataset

```
# Cell 7 — Convert pandas to DatasetDict
dataset = DatasetDict({
    "train": Dataset.from_pandas(train_df.reset_index(drop=True)),
    "test": Dataset.from_pandas(test_df.reset_index(drop=True))
})

dataset
```

```
DatasetDict({
    train: Dataset({
        features: ['model_text', 'label'],
        num_rows: 35918
    })
    test: Dataset({
        features: ['model_text', 'label'],
        num_rows: 8980
    })
})
```

Load Tokenizer

```
# Cell 8 — Tokenizer
model_checkpoint = "distilbert-base-uncased"
tokenizer = AutoTokenizer.from_pretrained(model_checkpoint)
print("Tokenizer loaded.")
```

```
/usr/local/lib/python3.12/dist-packages/huggingface_hub/utils/_auth.py:94: UserWarning:
The secret `HF_TOKEN` does not exist in your Colab secrets.
To authenticate with the Hugging Face Hub, create a token in your settings tab (https://huggingface.co/settings/tokens), set
You will be able to reuse this secret in all of your notebooks.
Please note that authentication is recommended but still optional to access public models or datasets.
  warnings.warn(
```

config.json: 100% 483/483 [00:00<00:00, 14.1kB/s]

tokenizer_config.json: 100% 48.0/48.0 [00:00<00:00, 2.16kB/s]

```
Warning: You are sending unauthenticated requests to the HF Hub. Please set a HF_TOKEN to enable higher rate limits and faste
WARNING:huggingface_hub.utils._http:Warning: You are sending unauthenticated requests to the HF Hub. Please set a HF_TOKEN to
```

vocab.txt: 232k/? [00:00<00:00, 2.49MB/s]

tokenizer.json: 466k/? [00:00<00:00, 2.80MB/s]

```
Tokenizer loaded.
```

Tokenize Dataset

```
# Cell 9 — Tokenisation
def tokenize_function(batch):
    return tokenizer(
        batch["model_text"],
        truncation=True,
        padding="max_length",
        max_length=256
    )

tokenized_dataset = dataset.map(tokenize_function, batched=True)

# Remove raw text column
tokenized_dataset = tokenized_dataset.remove_columns(["model_text"])

tokenized_dataset
```

```
Map: 100%                                          35918/35918 [00:33<00:00, 870.41 examples/s]

Map: 100%                                          8980/8980 [00:10<00:00, 919.47 examples/s]

DatasetDict({
    train: Dataset({
        features: ['label', 'input_ids', 'token_type_ids', 'attention_mask'],
        num_rows: 35918
    })
    test: Dataset({
        features: ['label', 'input_ids', 'token_type_ref', 'attention_mask'],
        num_rows: 8980
    })
})
```

Load DistilBert Model

```
# Cell 10 — Load model
model = AutoModelForSequenceClassification.from_pretrained(
    model_checkpoint,
    num_labels=2
)

print("Model loaded.")
```

```
model.safetensors: 100%                                    268M/268M [00:02<00:00, 215MB/s]

Loading weights: 100%                    100/100 [00:00<00:00, 283.17it/s, Materializing param=distilbert.transformer.layer.5.sa_layer_norm.weight]

DistilBertForSequenceClassification LOAD REPORT from: distilbert-base-uncased
Key                     | Status     |
------------------------+------------+-
vocab_transform.bias    | UNEXPECTED |
vocab_layer_norm.bias   | UNEXPECTED |
vocab_projector.bias    | UNEXPECTED |
vocab_layer_norm.weight | UNEXPECTED |
vocab_transform.weight  | UNEXPECTED |
pre_classifier.weight   | MISSING    |
classifier.weight       | MISSING    |
pre_classifier.bias     | MISSING    |
classifier.bias         | MISSING    |

Notes:
- UNEXPECTED    :can be ignored when loading from different task/architecture; not ok if you expect identical arch.
- MISSING   :those params were newly initialized because missing from the checkpoint. Consider training on your downstream ta
Model loaded.
```

Data Collator

```
# Cell 11 — Data collator
data_collator = DataCollatorWithPadding(tokenizer=tokenizer)
```

Define evolution Metrics

```
# Cell 12 — Metrics
def compute_metrics(eval_pred):
    logits, labels = eval_pred
    predictions = np.argmax(logits, axis=-1)
```

```
    precision, recall, f1, _ = precision_recall_fscore_support(
        labels, predictions, average="binary"
    )
    accuracy = accuracy_score(labels, predictions)

    return {
        "accuracy": accuracy,
        "precision": precision,
        "recall": recall,
        "f1": f1
    }

print("Metrics ready.")
```

```
Metrics ready.
```

Training Arguments

```
# Cell 13 — Training settings
training_args = TrainingArguments(
    output_dir="./distilbert_fake_news",
    eval_strategy="epoch",
    save_strategy="epoch",
    learning_rate=2e-5,
    per_device_train_batch_size=8,
    per_device_eval_batch_size=8,
    num_train_epochs=2,
    weight_decay=0.01,
    load_best_model_at_end=True,
    metric_for_best_model="f1",
    fp16=torch.cuda.is_available(),
    logging_steps=100,
    report_to="none"
)

print("Training arguments set.")
```

```
Training arguments set.
```

Trainer

```
trainer = Trainer(
    model=model,
    args=training_args,
    train_dataset=tokenized_dataset["train"],
    eval_dataset=tokenized_dataset["test"],
    data_collator=data_collator,
    compute_metrics=compute_metrics
)
```

```
# Cell 15 — Train model
trainer.train()
```

[8980/8980 11:34, Epoch 2/2]

Epoch	Training Loss	Validation Loss	Accuracy	Precision	Recall	F1
1	0.000065	0.001992	0.999666	1.000000	0.999300	0.999650
2	0.014920	0.003443	0.999332	0.999067	0.999533	0.999300

Writing model shards: 100% 1/1 [00:07<00:00, 7.26s/it]

Writing model shards: 100% 1/1 [00:01<00:00, 1.54s/it]

```
There were missing keys in the checkpoint model loaded: ['distilbert.embeddings.LayerNorm.weight', 'distilbert.embeddings.Lay
There were unexpected keys in the checkpoint model loaded: ['distilbert.embeddings.LayerNorm.beta', 'distilbert.embeddings.La
TrainOutput(global_step=8980, training_loss=0.0062914270351945275, metrics={'train_runtime': 696.6003,
'train_samples_per_second': 103.124, 'train_steps_per_second': 12.891, 'total_flos': 4757964024926208.0, 'train_loss':
0.0062914270351945275, 'epoch': 2.0})
```

evalute model

```
# Cell 16 — Final evaluation
results = trainer.evaluate()
print(results)
```

[1123/1123 00:27]

```
{'eval_loss': 0.001991138095036149, 'eval_accuracy': 0.9996659242761693, 'eval_precision': 1.0, 'eval_recall': 0.999299719887
```

detailed classification report

```
# Cell 17 — Full classification report
pred_output = trainer.predict(tokenized_dataset["test"])
y_pred = np.argmax(pred_output.predictions, axis=-1)
y_true = pred_output.label_ids

print(classification_report(y_true, y_pred))
```

```
              precision    recall  f1-score   support

           0       1.00      1.00      1.00      4696
           1       1.00      1.00      1.00      4284

    accuracy                           1.00      8980
   macro avg       1.00      1.00      1.00      8980
weighted avg       1.00      1.00      1.00      8980
```

save model and tokenizer

```
# Cell 18 — Save trained model
save_path = "./distilbert_fake_news_final"

trainer.save_model(save_path)
tokenizer.save_pretrained(save_path)

print("Model and tokenizer saved to:", save_path)
```

Writing model shards: 100% 1/1 [00:01<00:00, 1.10s/it]

```
Model and tokenizer saved to: ./distilbert_fake_news_final
```

Fake news prediction function

```
# Cell 19 — Prediction function
device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
model.to(device)

def predict_fake_news(text):
    inputs = tokenizer(
        text,
        return_tensors="pt",
        truncation=True,
        padding=True,
        max_length=256
    )

    inputs = {k: v.to(device) for k, v in inputs.items()}

    with torch.no_grad():
        outputs = model(**inputs)

    probs = torch.softmax(outputs.logits, dim=1)
    pred_class = torch.argmax(probs, dim=1).item()
    confidence = probs[0][pred_class].item()

    label = "Fake" if pred_class == 0 else "Real"
    return label, confidence
```

Testing custom text

```
# Cell 20 — Quick manual test
sample_text = "Breaking news: government announces a new healthcare reform plan for next year."
label, confidence = predict_fake_news(sample_text)

print("Prediction:", label)
print("Confidence:", round(confidence, 4))
```

```
Prediction: Fake
Confidence: 0.9986
```

connecting to newsAPI

```
# Cell 21 — NewsAPI live detection
import requests

api_key = "e0e70dade33d46599e66beafd6658eed"
url = f"https://newsapi.org/v2/top-headlines?language=en&pageSize=10&apiKey={api_key}"

response = requests.get(url)
data = response.json()

if data.get("status") != "ok":
    print("Error from NewsAPI:", data)
else:
    print("Live News Detection with DistilBERT\n")

    for article in data.get("articles", []):
        title = article.get("title", "")
        description = article.get("description", "")

        # combine title + description for better prediction
        text_input = (title + " " + str(description)).strip()

        if text_input:
            start = time.time()
            label, confidence = predict_fake_news(text_input)
            end = time.time()

            print("Headline:", title)
            print("Prediction:", label)
            print("Confidence:", round(confidence, 4))
            print("Time:", round(end - start, 4), "seconds\n")
```

```
Live News Detection with DistilBERT

Headline: Laden Iranian ships depart Chinese port tied to key military chemicals - The Washington Post
Prediction: Real
Confidence: 1.0
Time: 0.0699 seconds

Headline: Kentucky Basketball loses to Florida on Senior Day: 3 things to know and postgame boos - A Sea Of Blue
Prediction: Fake
Confidence: 0.9995
Time: 0.0142 seconds

Headline: Trump says Iran at fault for strike on girls school - Politico
Prediction: Fake
Confidence: 0.9969
Time: 0.0122 seconds

Headline: Times the U.S. has installed a foreign leader, as Trump zeroes in on Iran - Axios
Prediction: Fake
Confidence: 0.9886
Time: 0.0125 seconds

Headline: Dan Hurley Loses His Mind, Gets Ejected With One Second Left Against Marquette - OutKick
Prediction: Fake
Confidence: 0.9993
Time: 0.0127 seconds

Headline: Shedding light on how Epstein used visits to Interlochen to target girls - NPR
Prediction: Fake
Confidence: 0.9946
Time: 0.0122 seconds

Headline: Live results and analysis for UFC 326: Holloway vs. Oliveira 2 - ESPN
Prediction: Real
```

```
Confidence: 0.9834
Time: 0.0096 seconds

Headline: Health officials confirm measles case in visitor to Hawaii - Honolulu Star-Advertiser
Prediction: Real
Confidence: 0.5234
Time: 0.0129 seconds

Headline: Rapper-politician Balendra Shah on course to be Nepal's next prime minister - BBC
Prediction: Real
Confidence: 0.9999
Time: 0.0121 seconds

Headline: Enormous 3D map of the universe shows brilliant 'sea of light' near the cosmic dawn - Live Science
Prediction: Fake
Confidence: 0.9976
Time: 0.0134 seconds
```